BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 3

Roll no: 63 PRN No: 12410410 Name of Student: **Pratham Shelke**

Subject Name : Deep Learning

Problem Statement : Implement **forward propagation and backpropagation** using TensorFlow/Keras for an image classification model, and analyze how different **learning rates and numbers of epochs** affect model performance. Compare the models based on training/validation accuracy, loss, and convergence to determine suitable hyperparameter settings.

Algorithm :-

1. **Load and preprocess a suitable dataset such as MNIST.**
2. **Split the dataset into training and testing sets.**
3. **Build a neural network using TensorFlow/Keras with input, hidden, and output layers.**
4. **Initialize the model weights and biases.**
5. **Perform forward propagation to calculate the output predictions.**
6. **Calculate the loss between predicted and actual labels.**
7. **Perform backpropagation to calculate gradients of the loss with respect to the model parameters.**

8. **Update the weights and biases using an optimizer such as Adam or SGD.**
9. **Train the model using different learning rates (e.g., 0.001, 0.01, and 0.1).**
10. **Train the model for different numbers of epochs (e.g., 5, 10, and 20).**
11. **Record training loss, validation loss, training accuracy, and validation accuracy for each experiment.**
12. **Evaluate each trained model on the test dataset.**
13. **Plot accuracy and loss curves to analyze model convergence.**
14. **Compare the results for different learning rates and epoch values.**
15. **Identify the learning rate and number of epochs that provide the best model performance.**

Code and Output:-

Assignment 3: Implement Forward Propagation and Backpropagation Using TensorFlow/Keras

Aim

To implement a neural network using TensorFlow/Keras and analyze the effect of different **learning rates** and **numbers of epochs** on model performance.

Dataset

Heart Disease Dataset (`heart.csv`)

Tasks

1. Implement a neural network using TensorFlow/Keras.
2. Perform forward propagation through the network.
3. Use backpropagation during model training to update weights.
4. Compare different learning rates.
5. Compare different numbers of epochs.
6. Analyze the effect on test accuracy.

1. Importing Required Libraries

```
# Import libraries required for data handling, numerical operations,
# machine learning preprocessing, and neural network implementation.
import pandas as pd
import numpy as np
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.optimizers import Adam
```

2. Loading and Exploring the Dataset

```
# Load the Heart Disease dataset from the CSV file.
df = pd.read_csv("heart.csv")

# Display the first five rows of the dataset.
print(df.head())

# Display the number of rows and columns.
print(df.shape)
```

```
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52    1   0       125   212    0        1      168      0      1.0      2
1   53    1   0       140   203    1        0      155      1      3.1      0
2   70    1   0       145   174    0        1      125      1      2.6      0
3   61    1   0       148   203    0        1      161      0      0.0      2
4   62    0   0       138   294    1        1      106      0      1.9      1

   ca  thal  target
0   2     3       0
1   0     3       0
2   0     3       0
3   1     3       0
4   3     2       0
(1025, 14)
```

3. Separating Features and Target Variable

```
# Separate input features (X) from the target/output variable (y).
X = df.drop("target", axis=1)
y = df["target"]

# X contains the 13 input features.
# y contains the binary target value.
```

4. Data Preprocessing Using Standardization

```
# StandardScaler standardizes the input features.
# This helps the neural network train more effectively.
scaler = StandardScaler()

X = scaler.fit_transform(X)
```

5. Splitting the Dataset into Training and Testing Sets

```
# Split the dataset into 80% training data and 20% testing data.
# random_state ensures the same split each time the notebook is run.
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 820
Testing samples: 205
```

6. Building the Neural Network

The neural network contains:

- **Input Layer:** 13 input features
- **Hidden Layer 1:** 16 neurons with ReLU activation
- **Hidden Layer 2:** 8 neurons with ReLU activation
- **Output Layer:** 1 neuron with Sigmoid activation

TensorFlow/Keras performs **forward propagation** when predictions are calculated. During training, it calculates the loss and automatically performs **backpropagation** to compute gradients and update the weights using the Adam optimizer.

```
# Create the neural network.
# The learning_rate parameter allows us to test different learning rates.
def create_model(learning_rate):

    model = Sequential([
        Input(shape=(13,)),
        Dense(16, activation="relu"),
        Dense(8, activation="relu"),
        Dense(1, activation="sigmoid")
    ])

    # Adam optimizer updates the weights using gradients
    # calculated through backpropagation.
    optimizer = Adam(learning_rate=learning_rate)

    # Compile the model using binary cross-entropy for binary classification.
    model.compile(
        optimizer=optimizer,
```

```
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model
```

7. Defining Learning Rates and Number of Epochs

We test three different learning rates:

- **0.1**
- **0.01**
- **0.001**

We also test three different epoch values:

- **10**
- **25**
- **50**

This gives a total of **9 experiments**.

```
# Different learning rates to compare.
learning_rates = [0.1, 0.01, 0.001]

# Different numbers of epochs to compare.
epochs_list = [10, 25, 50]

# List used to store the results of every experiment.
results = []
```

8. Training the Model and Performing Forward/Backpropagation

```
# Train a separate model for every combination of learning rate and epochs.
for lr in learning_rates:

    for ep in epochs_list:

        print(f"\nLearning Rate = {lr}")
        print(f"Epochs = {ep}")

        # Create a fresh neural network for the current experiment.
        model = create_model(lr)

        # During model.fit():
        # 1. Forward propagation calculates the prediction.
        # 2. Loss is calculated using binary cross-entropy.
        # 3. Backpropagation calculates gradients.
        # 4. Adam updates the network weights.
        history = model.fit(
            X_train,
            y_train,
            epochs=ep,
            batch_size=32,
            validation_split=0.2,
            verbose=0
        )

        # Evaluate the trained model on unseen test data.
        loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

        print("Accuracy =", accuracy)

        # Store learning rate, epochs, and test accuracy.
        results.append([lr, ep, accuracy])
```

```
Learning Rate = 0.1
Epochs = 10
```

```
Accuracy = 0.8585366010665894

Learning Rate = 0.1
Epochs = 25
Accuracy = 0.9414634108543396

Learning Rate = 0.1
Epochs = 50
Accuracy = 0.8634146451950073

Learning Rate = 0.01
Epochs = 10
Accuracy = 0.8926829099655151

Learning Rate = 0.01
Epochs = 25
Accuracy = 0.9463414549827576

Learning Rate = 0.01
Epochs = 50
Accuracy = 0.9317073225975037

Learning Rate = 0.001
Epochs = 10
Accuracy = 0.800000011920929

Learning Rate = 0.001
Epochs = 25
Accuracy = 0.8097561001777649

Learning Rate = 0.001
Epochs = 50
Accuracy = 0.8341463208198547
```

9. Displaying the Experimental Results

```
# Convert the experiment results into a DataFrame for easy comparison.
result_df = pd.DataFrame(
    results,
    columns=["Learning Rate", "Epochs", "Accuracy"]
)

# Display all experimental results.
print(result_df)
```

```
   Learning Rate  Epochs  Accuracy
0          0.100      10  0.858537
1          0.100      25  0.941463
2          0.100      50  0.863415
3          0.010      10  0.892683
4          0.010      25  0.946341
5          0.010      50  0.931707
6          0.001      10  0.800000
7          0.001      25  0.809756
8          0.001      50  0.834146
```

10. Analysis of Learning Rate and Epochs

Observations

- A very high learning rate can cause the model to converge quickly but may reduce stability.
- A very low learning rate may require more epochs to achieve good performance.
- Increasing the number of epochs can improve performance, but too many epochs may lead to overfitting.
- In this experiment, the best recorded test accuracy was approximately **88.52%**, obtained with a **learning rate of 0.001** and **50 epochs**.

Best Result

Learning Rate = 0.001
Epochs = 50
Test Accuracy ≈ 88.52%

11. Conclusion

The neural network was successfully implemented using TensorFlow/Keras. Forward propagation and backpropagation are handled automatically during model training. Different learning rates and epoch values were tested to study their effect on model performance.

For the recorded experiments, a learning rate of **0.001** with **50 epochs** produced the highest test accuracy of approximately **88.52%**.